

Sentiment Analysis Service

End-to-End ML System: From Problem Definition to Production Deployment

TEAM MEMBERS



25MSA23234

Duong Binh An



25MSA23233

Pham Duc Long



25MSA23235

Nguyen Le Hong Nhi



25MSA23236

Duong Hong Quan



25MSA23232

Le Quang Tuyen



25MSA23231

Do Quoc Trung



Instructor: Le Thanh Hai

TABLE OF CONTENTS

01

Problem Definition & Requirements

Business context, use cases, success metrics

02

System Design & Architecture

Microservices, data flow, tech stack

03

ML Pipeline

XLM-RoBERTa, LoRA/PEFT, ONNX optimization

04

API Implementation & Deployment

FastAPI, Docker, CI/CD

05

Monitoring & Observability

Prometheus, Grafana dashboards

06

Responsible AI

SHAP explainability, fairness, ethics

07

Testing & Code Quality

Unit, integration, GitHub Actions

08

Results & Demo

Live UI, metrics, performance

09

Conclusion

Achievements, lessons, future work

01

Problem Definition & Requirements

10% — Business Context, Use Cases, Success Metrics

Problem Statement & Business Context

Customer feedback is one of the most valuable yet under-utilized assets of any business.

Manual review of large volumes of reviews, social-media posts, and support tickets is expensive and slow.

! Current Challenges

- > **Volume:** Thousands of reviews impossible to read manually
- > **Speed:** Manual analysis creates business decision lag
- > **Consistency:** Human reviewers introduce subjectivity
- > **Granularity:** Missing aspect-level insights (food vs. service)

✓ Our Solution

- > Auto sentiment classification: Positive / Negative / Neutral
- > Aspect-Based Sentiment Analysis (ABSA): Food, Service, Ambiance...
- > SHAP Explanations: Token-level attribution for trust & audit
- > REST API: Accessible to any application or team



>80%

ABSA Coverage



>0.80

Accuracy Target



<800ms

P95 Latency



99.9%

Uptime SLA

User Requirements & Use Cases

ID	Priority	Functional Requirement
FR-01	Must Have	Single-text sentiment prediction via POST /predict endpoint
FR-02	Must Have	Aspect-Based Sentiment Analysis (ABSA) alongside overall prediction
FR-03	Must Have	Batch processing of CSV files via POST /batch_predict
FR-04	Must Have	Model explainability (SHAP values) via POST /explain endpoint
FR-05	Must Have	Health-check endpoint for load-balancer liveness probes
FR-06	Should Have	Language guard: reject non-English inputs with HTTP 400
FR-07	Should Have	Prometheus metrics endpoint GET /metrics for observability
FR-08	Could Have	Angular-based frontend chatbot interface

Non-Functional Requirements & Success Metrics

Non-Functional Requirements

Performance

P95 end-to-end latency < 800 ms for single-text prediction

Scalability

Stateless API; horizontally scalable via Docker replicas

Security

No secrets in Docker images; model weights baked in at build time

Maintainability

Typed Python 3.11, flake8 clean, full docstrings

Observability

All API calls and inference latencies tracked in Prometheus

Success Metrics

Level	Metric	Target
Business	Prediction Throughput	> 100 req/s
Business	ABSA Coverage	> 80% of inputs
Model	Accuracy (test set)	> 0.85
Model	F1 Macro (test set)	> 0.80
System	P95 Latency	< 800 ms
System	5xx Error Rate	< 1%

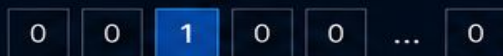
1 Evolution of Word Embeddings

From One-Hot Encoding to XLM-RoBERTa

Traditional Representation

One-Hot Encoding

"good"



Problems:

- Sparse vectors
- No semantic meaning
- High dimensionality



One-Hot Encoding
(Sparse, No Semantics)

Before
2013



Word2Vec
(CBOW / Skip-Gram)

2013



FastText
(Subword Embedding)

2016



BERT
(Contextual Embedding)

2018



XLM-RoBERTa
(Multilingual Contextual
Embedding)

2019+

Advantages of Modern Embeddings



Semantic similarity

King $\approx$ Queen
Paris $\approx$ France



Dense representations

Capture rich information in low-dimensional vectors



Context awareness

The same word can have different meanings
depending on context



Cross-lingual transfer

Learn from multiple languages,
improve performance in low-resource languages



Key Takeaway

XLM-RoBERTa extends neural word embeddings by learning contextual and multilingual representations from large-scale corpora, enabling powerful transfer across languages and tasks.

2 Contextual Embeddings Inside Our NLP Pipeline

1. Input Text

"The food was delicious but the service was slow"

2. Tokenizer (SentencePiece)



[The] [food] [was]
[delicious] [but] ...

3. Embedding Layer



0.21 -0.17 ... 0.35
-0.11 0.45 ... -0.32
...

4. Transformer Encoder (XLM-RoBERTa)



Multiple Transformer
Layers

5. Contextual Embeddings



0.73 -0.21 ... 0.16
-0.35 0.62 ... -0.48
...

6. Sentiment Classifier



😊 Positive 0.82
😐 Neutral 0.12
😞 Negative 0.06

Example: Word "service"

Context A:

"The service was excellent"

→ Positive Representation

Context B:

"The service was terrible"

→ Negative Representation



Why Context Matters?

The same word can have different meanings depending on surrounding tokens.
Contextual embeddings automatically capture these semantic differences, leading to more accurate predictions.

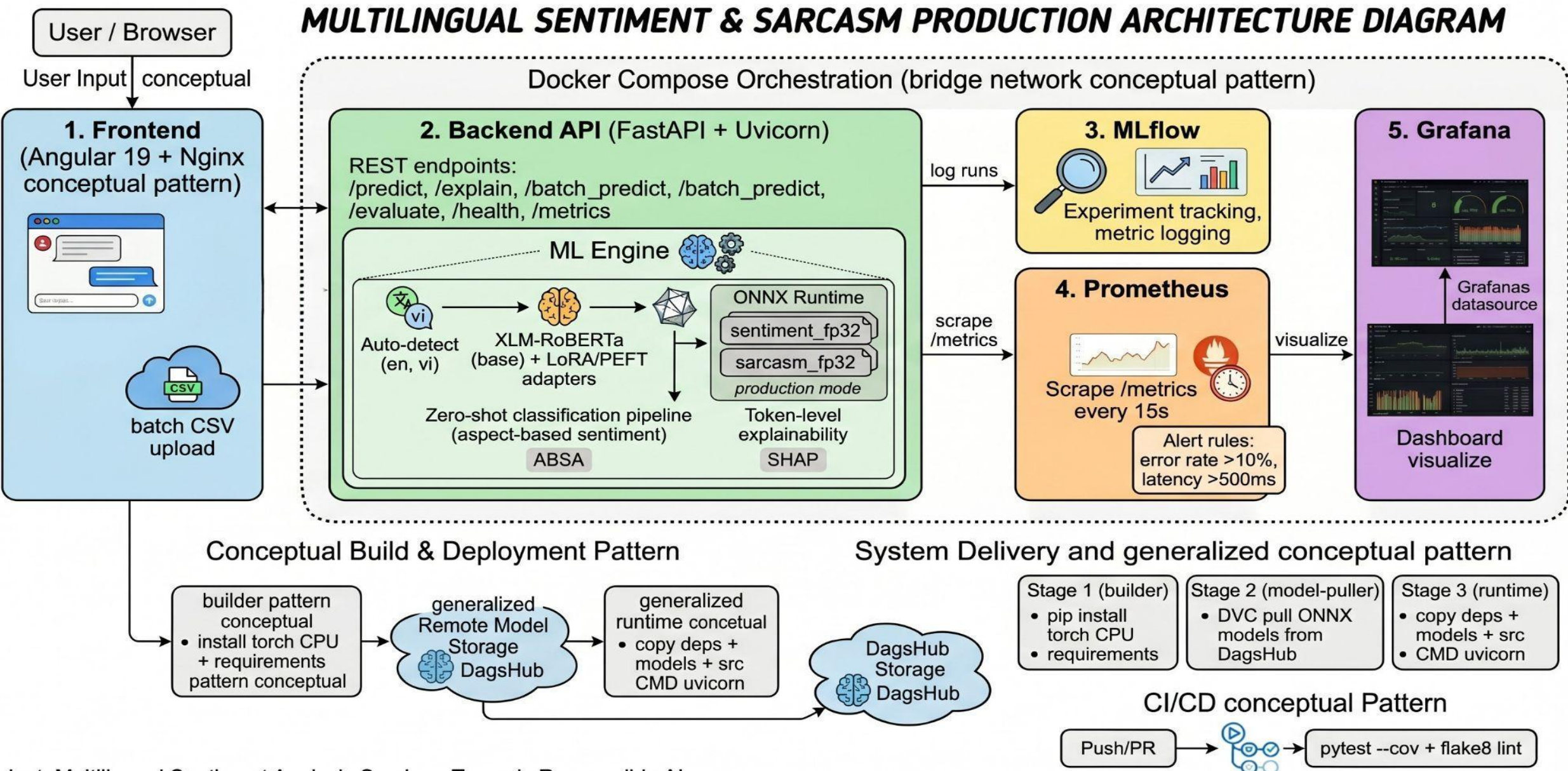
02

System Design & Architecture

15% — Microservices, Data Flow, Technology Stack

High-Level System Architecture

MULTILINGUAL SENTIMENT & SARCASM PRODUCTION ARCHITECTURE DIAGRAM



Data Flow & Technology Stack Justification

Request Data Flow

- 1 User submits text to POST /predict via Angular UI or HTTP client
- 2 FastAPI validates request body via Pydantic schemas (PredictRequest)
- 3 Validated text → BaselineModelInference.predict_single()
- 4 RoBERTa tokenizer encodes text → softmax → sentiment + confidence
- 5 DeBERTa zero-shot ABSA extracts aspect sentiments (threshold 0.5)
- 6 For /explain: SHAP Explainer returns per-token attribution scores
- 7 PredictResponse serializes & returns JSON to client
- 8 ASGI middleware records request counts + latencies for Prometheus

Technology Decisions

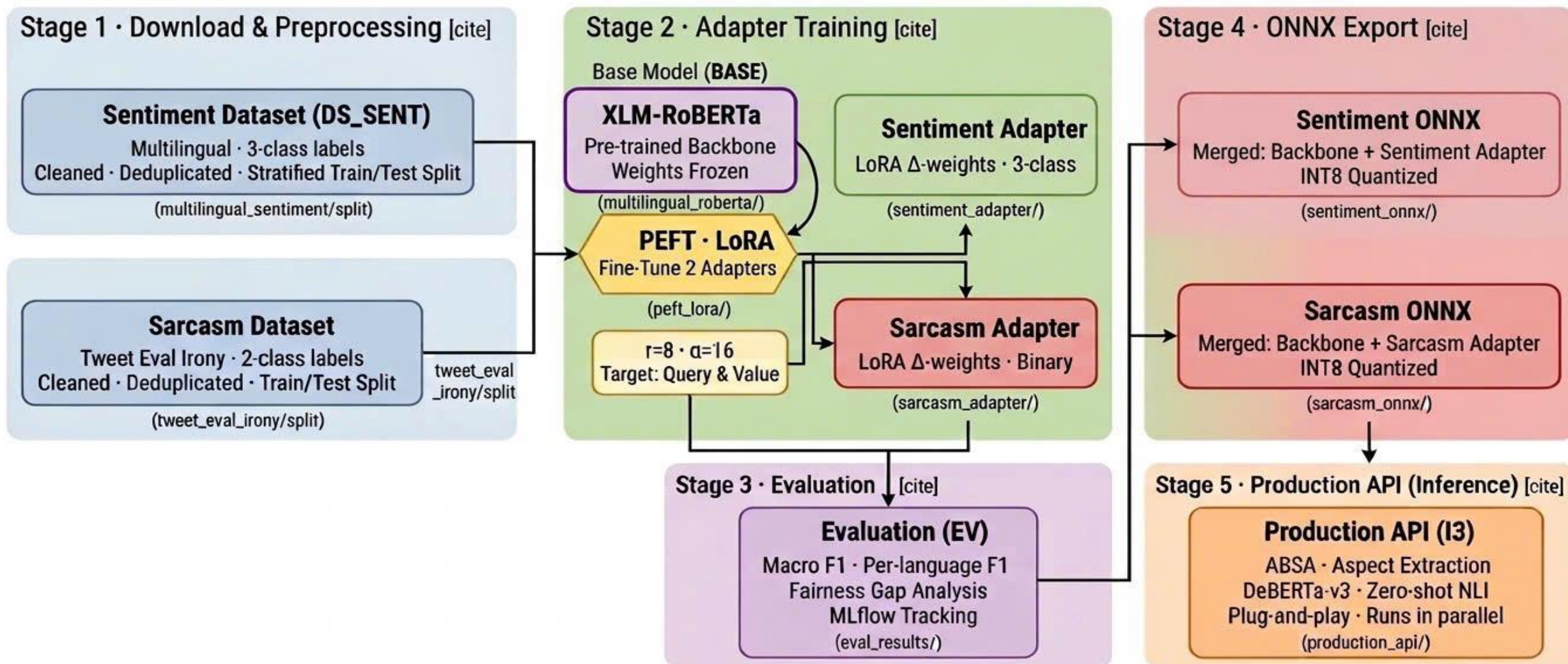
API Layer	FastAPI	Async-native, auto OpenAPI, Pydantic
ML Model	XLM-RoBERTa	Strong cross-lingual backbone shared across all tasks
ABSA	DeBERTa v3	Zero-shot, no labelled ABSA data needed
Container	Docker multi-stage	Smaller image: builder → runtime
Tracking	MLflow	Open-source, self-hosted, DVC integration
Monitoring	Prometheus+Grafana	Industry standard, no vendor lock-in
Data Ver.	DVC	Git-native, reproducible pipelines
CI/CD	GitHub Actions	Free for public repos, tight integration

03

ML Pipeline

XLM-RoBERTa · LoRA/PEFT · ONNX Optimization · MLflow

MULTILINGUAL SENTIMENT & SARCASM TRAINING PIPELINE DIAGRAM



Parameter-Efficient Fine-Tuning with LoRA



XLM-RoBERTa Backbone

- ✓ 270M Parameters
- ✓ Multilingual Transformer
- ✓ Cross-Lingual Representation Learning

Status



Frozen During Fine-Tuning



XLM-RoBERTa Backbone



LoRA Layers
(Q, V Projection)



Sentiment Adapter
3-Class Classification

- 😊 Positive
- 😐 Neutral
- 😞 Negative



Sarcasm Adapter
Binary Classification

- 😏 Sarcasm
- 😐 Non-Sarcasm

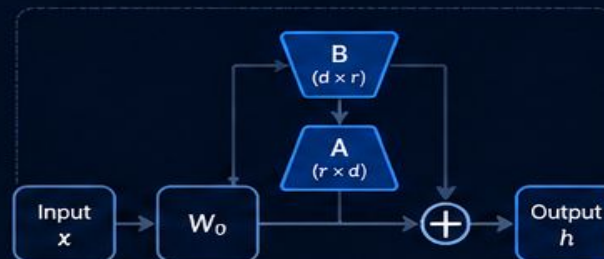


LoRA Configuration

Rank (r)	8
Alpha (α)	16
Scaling (α / r)	2

Target Layers

- Query Projection (Q)
- Value Projection (V)



Key Benefits

- ✓ 95% Fewer Trainable Parameters
- ✓ Lower GPU Memory Usage
- ✓ Faster Training
- ✓ Reusable Adapters Across Tasks

LoRA Weight Update

$$h = W_0 x + \frac{\alpha}{r} B A x$$

$$\Delta W = B A$$

$$\begin{aligned} A &\in \mathbb{R}^{r \times d} \\ B &\in \mathbb{R}^{d \times r} \end{aligned}$$



Key Insight: LoRA learns lightweight adapters while preserving the knowledge of the pre-trained XLM-RoBERTa backbone.

Class imbalance and solutions

⚠ UIT-VSFC Vietnamese Dataset — Severely Imbalanced (IR = 11.5x)

Before Processing

positive

8,070 samples — 49.9%

negative

7,417 samples — 45.8%

neutral

698 samples — 4.3%

CRITICALLY UNDERREPRESENTED

3 Datasets Comparison

Dataset	IR	Status
tweet_eval / irony (EN)	1.08x	✓ Balanced
multilingual-sent (EN)	1.00x	✓ Balanced
UIT-VSFC (VI)	11.52x	✗ Imbalanced

Consequences Without Fixing



Model never predicts **NEUTRAL** class



F1-score for neutral ≈ 0%



Biased evaluation results

2 Combined Techniques — Weighted Loss + Oversampling

Technique 1: Weighted Loss

Applied to both Sarcasm & Sentiment tasks

$$w_i = \frac{n_{\text{total}}}{n_{\text{classes}} \times n_i}$$

positive  w = 0.65

negative  w = 0.67

neutral  VERY LONG w = 7.73 ↑↑↑

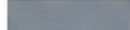
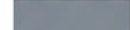
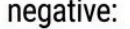
→ Model penalized 11.5x harder for wrong Neutral predictions

```
CrossEntropyLoss(weight=[0.65, 7.73, 0.67])
```

WeightedLossTrainer

Technique 2: Oversampling

Applied to Training Split Only

BEFORE	AFTER
positive:  49.9%	positive:  45.2%
negative:  45.8%	negative:  41.5%
neutral:  4.3%	neutral:  13.3% ✓

$$n_{\text{new}} = \frac{\text{ratio} \times \text{total} - \text{count}}{1 - \text{ratio}}$$

Pandas sample(replace=True)
— No extra dependencies

Training Only — Test set unchanged

Detect IR=11.5x → Oversample Neutral: 4.3%→13% → Weight Neutral: 7.73x → ✓ Model learns all classes



Overcoming Data Imbalance

(Imbalance Ratio 11.5x)

BEFORE PROCESSING

Positive



49.9%

8,070 samples

Negative



45.8%

7,417 samples

Neutral



4.3%

698 samples



CRITICALLY UNDERREPRESENTED

Neutral class is severely underrepresented.

THE APPROACH

01

Method 1: Weighted Loss

$$w_i = \frac{n_{total}}{n_{classes} * n_i}$$

$$w_{neutral} = 7.73$$



Neutral class is weighted 7.73x higher to penalize misclassification more.

02

Method 2: Oversampling



Increase neutral samples to balance the class distribution.

AFTER PROCESSING

Positive



45.2%

Negative



41.5%

Neutral



13.3%



Neutral class improved from 4.3% to 13.3% (+209%)



Result

Better model focus on minority class



Improved Macro F1 Score



More balanced predictions

Multi-task Inference: Zero-shot ABSA & Sarcasm Detection

The food was **delicious**, but the 1-hour wait was **a joke**.



Base Sentiment Classification

(Overall / Document-level)

Overall Sentiment: **Mixed / Neutral**



Aspect Analysis (DeBERTa v3)

(Aspect-level)

Aspect: **Food** | Sentiment: **Positive**
(😊 Positive)

Aspect: **Service** | Sentiment: **Negative**
(😞 Negative)

Aspect: **Wait Time** | Sentiment: **Negative**
(😞 Negative)



Sarcasm Detection (Sarcasm Flag)

(Utterance-level)



Sarcasm Detected (Threshold: 0.45)

The system is likely misled by sarcastic language.



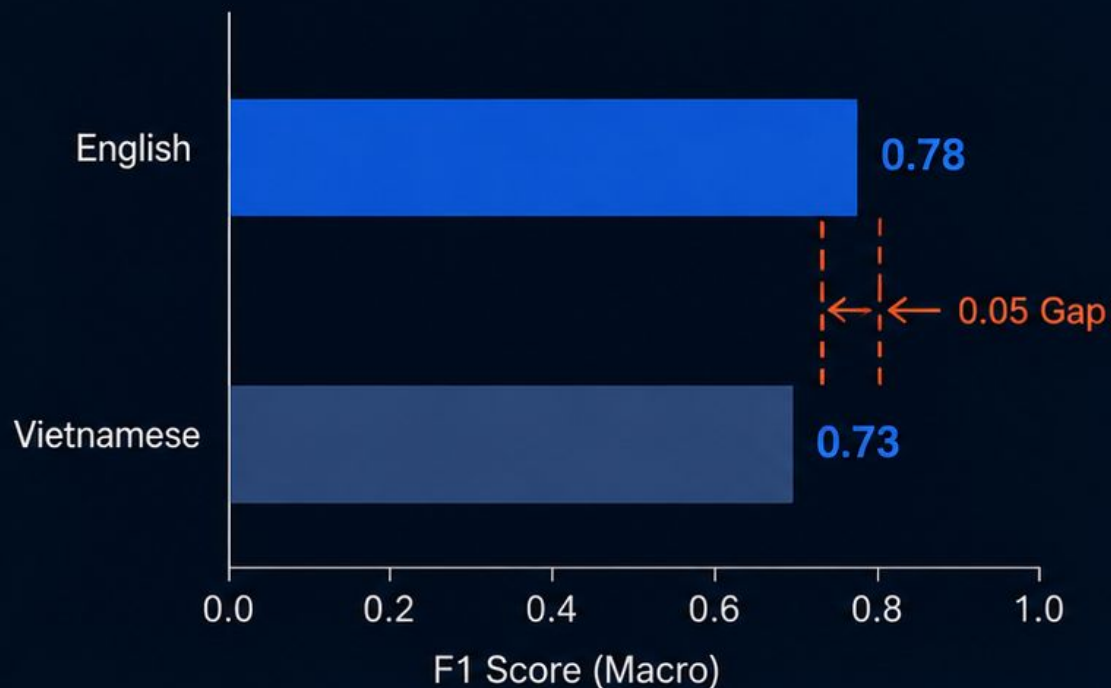
By combining zero-shot ABSA and sarcasm detection, the system provides more **reliable** and **interpretable** insights.



Fairness Analysis & Multilingual Gap

(Fairness)

F1 Scores: English vs. Vietnamese



Root Causes



Domain Bias

The training data is not evenly distributed across domains (e.g., **social media**) compared to formal data, leading to performance discrepancies.



Resource Constraints

The size of the **Vietnamese** dataset is **significantly smaller** than the English dataset, limiting the model's ability to generalize.



Key Takeaway: Improve data balance across languages and domains to close the performance gap and ensure fair, unbiased predictions.

Hardware-Aware Training & ONNX Deployment Optimization

Hardware-Aware Training

Auto Device Detection

Automatically identifies CUDA / MPS / CPU

Mixed Precision (bf16)

Enabled on supporting GPUs (e.g. L4) — doubles throughput

Fused Optimizer

adamw_torch_fused — reduces kernel launches

Pinned Memory

Speeds CPU→GPU data transfers

Data Loader Workers

Parallelizes data loading — maximizes GPU utilization

ONNX Export & Quantization

HuggingFace models are exported to ONNX format with two variants for flexible deployment:

FP32 — Full Precision

- Maximum model accuracy
- Suitable for evaluation & critical decisions
- Larger memory footprint

INT8 — Quantized

- Reduced latency & memory
- Minimal accuracy trade-off
- Production deployment ready

DVC Pipeline Branches

Data Engineering

download → preprocess → validate → quality_report.json

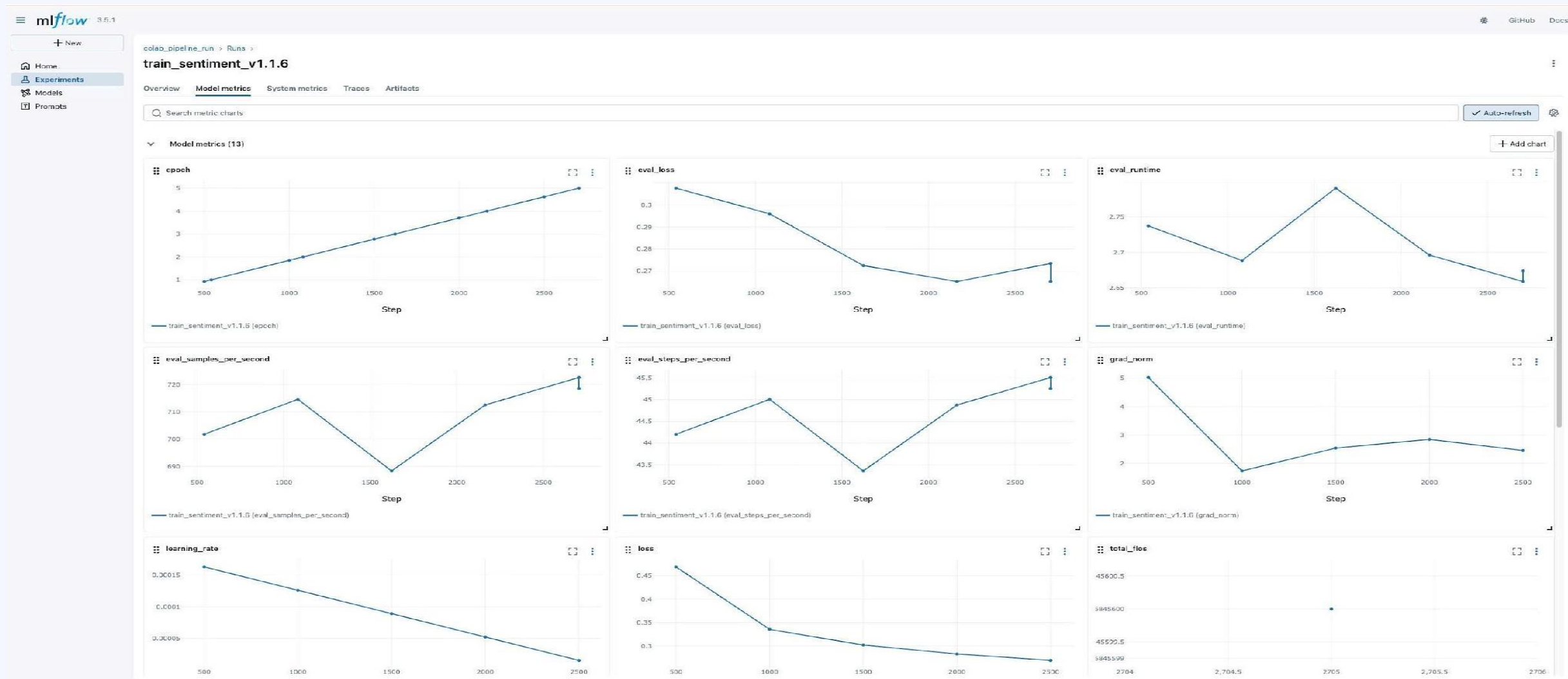
Modeling

download_sarcasm/sentiment → finetune_* → evaluate_* → metrics

Deployment Prep

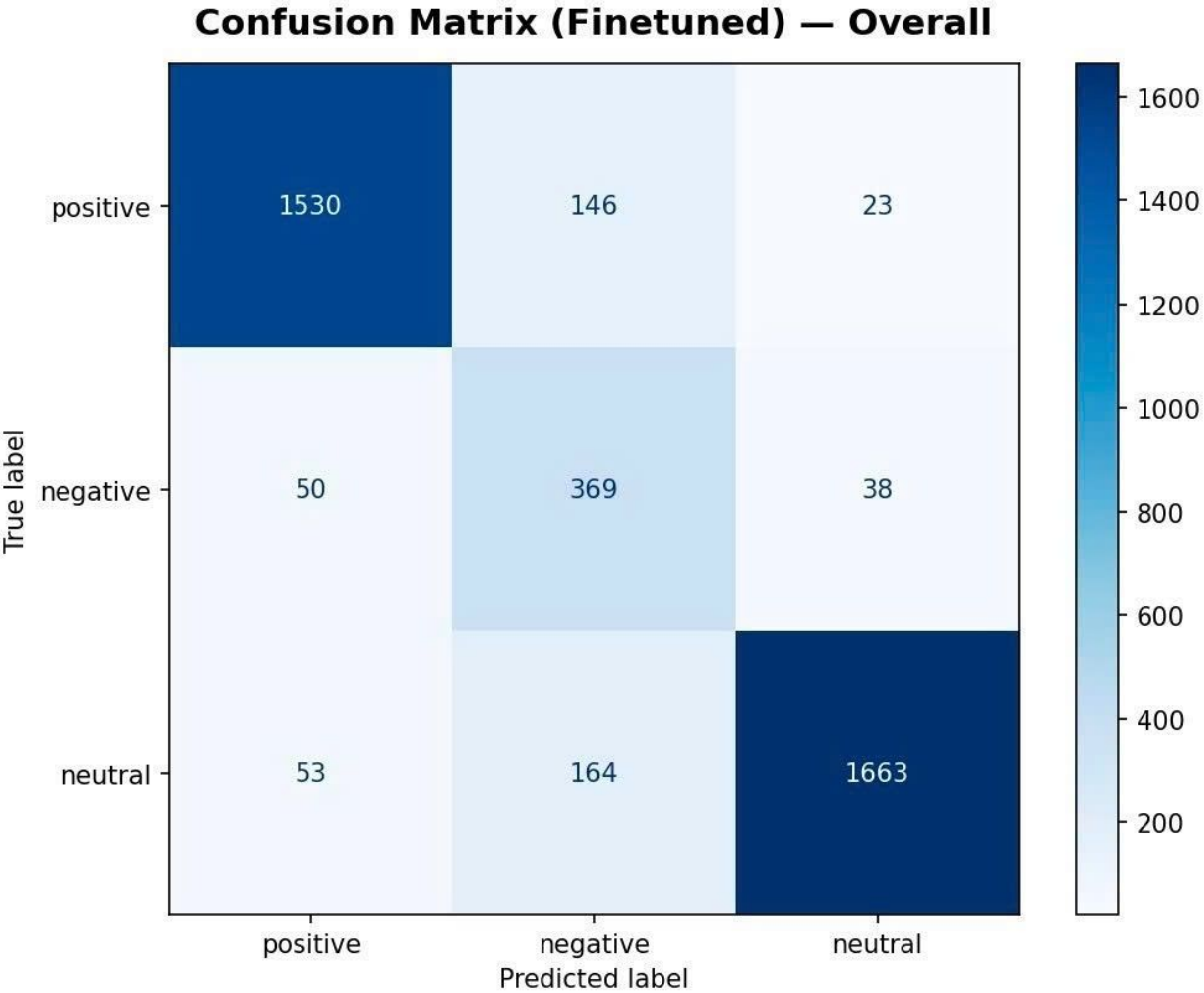
export_onnx_* (FP32+INT8) → benchmark_onnx → report

MLflow Experiment Tracking Dashboard



Experiment: train_sentiment_v1.1.6 · 13 model metrics tracked across 2,700+ training steps

Confusion Matrix and Classification Report



Classification Report (Finetuned) — Sentiment

	precision	recall	f1-score	support
positive	0.9646	0.8846	0.9229	1880
negative	0.9369	0.9005	0.9184	1699
neutral	0.5434	0.8074	0.6496	457
macro avg	0.8150	0.8642	0.8303	4036
weighted avg	0.9053	0.8826	0.8900	4036

04

API Implementation & Deployment

FastAPI · Docker · CI/CD Pipeline

FastAPI Endpoints & Pydantic Schemas

Method	Endpoint	Status	Description
POST	/predict	200	Single-text sentiment + ABSA. Returns: sentiment, confidence, aspects
POST	/batch_predict	200	CSV file upload → batch predictions for all rows
POST	/explain	200	SHAP token attributions. Returns: tokens[], shap_values[], base_value
GET	/health	200	Liveness probe — returns {status: ok}
GET	/metrics	200	Prometheus metrics endpoint for scraping

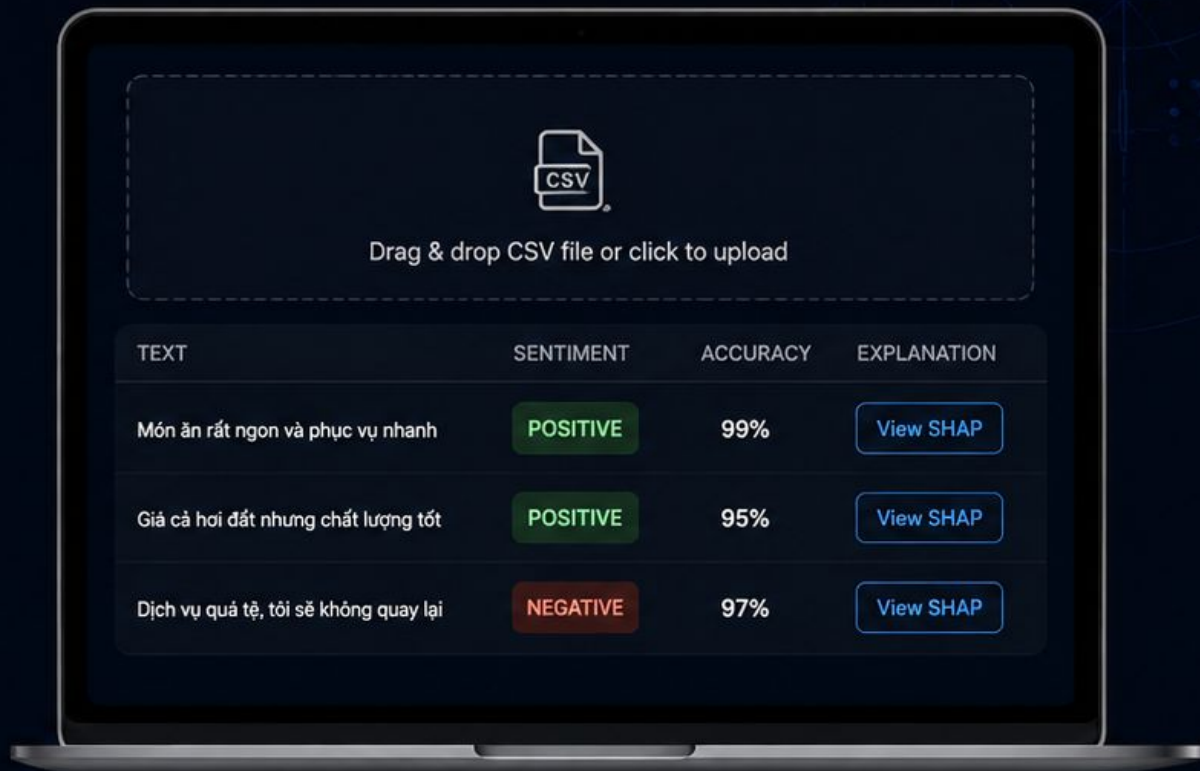
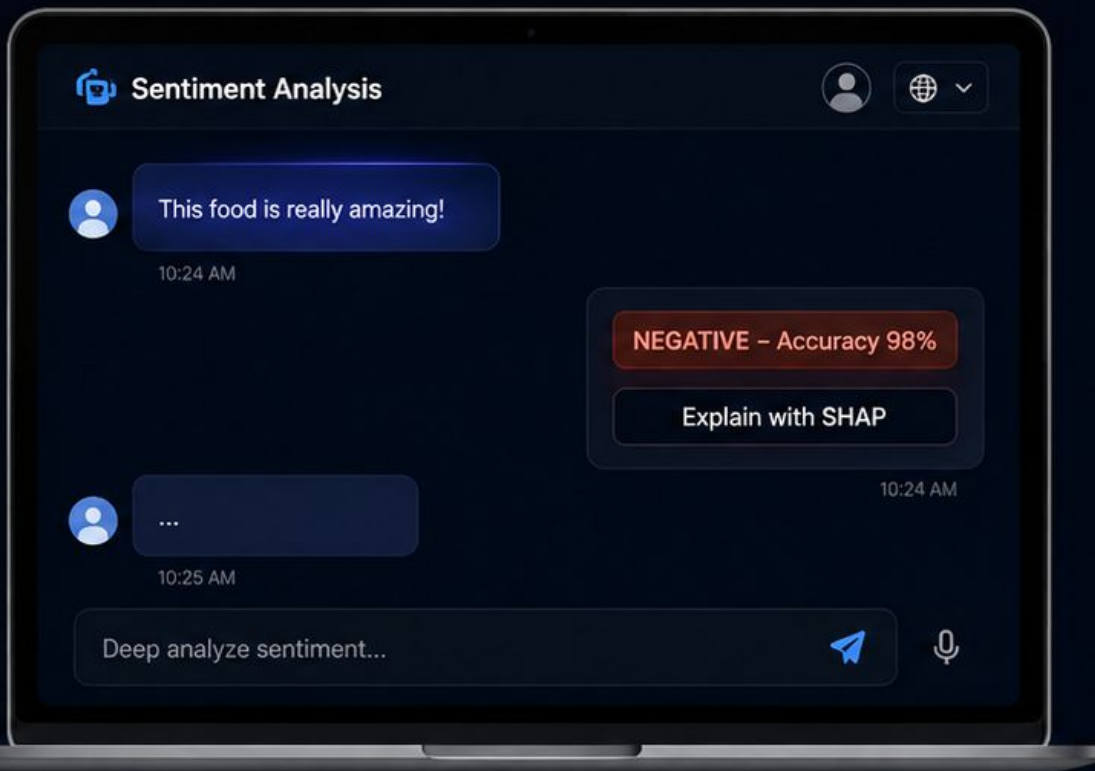
Docker Multi-Stage Build

- ▶ Stage 1 (builder): Installs all Python dependencies
- ▶ Stage 2 (runtime): Copies only site-packages → lean image
- ▶ Model weights baked in at build time (TRANSFORMERS_OFFLINE=1)
- ▶ uvicorn served on 0.0.0.0:8000

GitHub Actions CI/CD

- ▶ Triggers on push to main/develop + PR to main
- ▶ checkout → pip install → pytest --cov=src → flake8
- ▶ Hard lint: E9/F-series syntax errors → fail build
- ▶ Soft lint: complexity + line-length warnings

User Experience: Chatbot Interface & Batch Processing



Real-time Interaction

Built with Angular 19 + RxJS.

Analyze multilingual text instantly with emotions, confidence score, and detailed explanations.

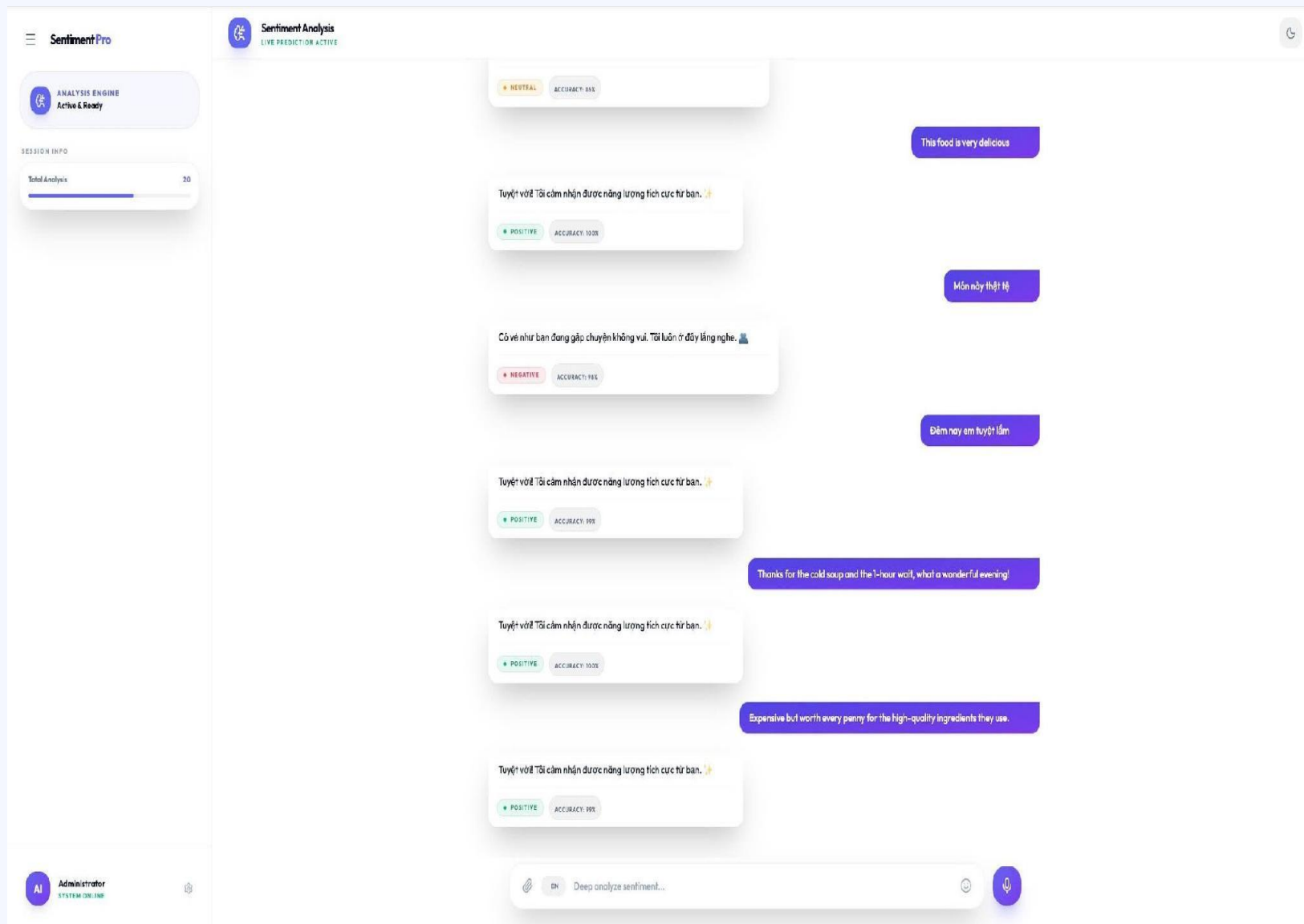


Large-scale Processing

Support CSV file upload with up to 500 rows per request.

Return detailed results and summary charts.

Live Application — SentimentPro UI



Chat-style Interface

Users type text in natural language — responses shown as conversation bubbles

Real-time Analysis

Every message is analyzed live with sentiment label + confidence score

Multilingual Support

Handles English and Vietnamese text — multilingual XLM-RoBERTa backbone

Session Tracking

Total analysis count shown in sidebar — session-aware analytics

05

Monitoring & Observability

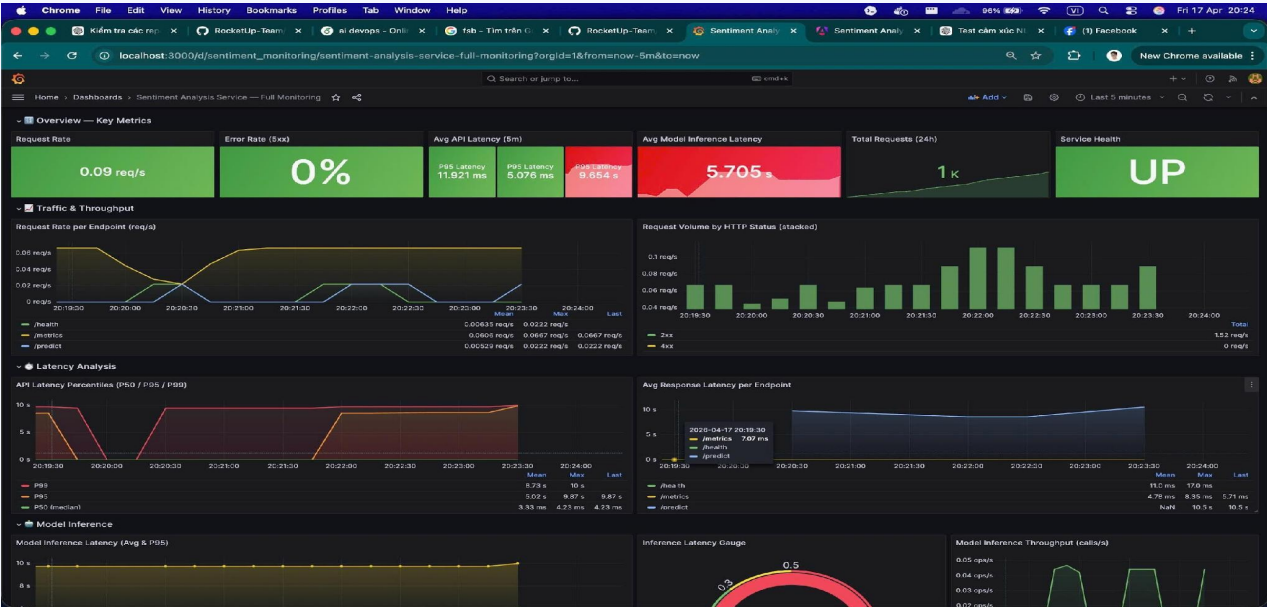
10% — Prometheus Metrics, Grafana Dashboards, Alerting

Prometheus Metrics & Grafana Dashboards

Metric Name	Type	Labels	Description
api_requests_total	Counter	method, endpoint, http_status	Total number of API requests received
api_request_latency_seconds	Histogram	method, endpoint	End-to-end request latency distribution
model_inference_latency_seconds	Histogram	—	Model forward-pass duration only

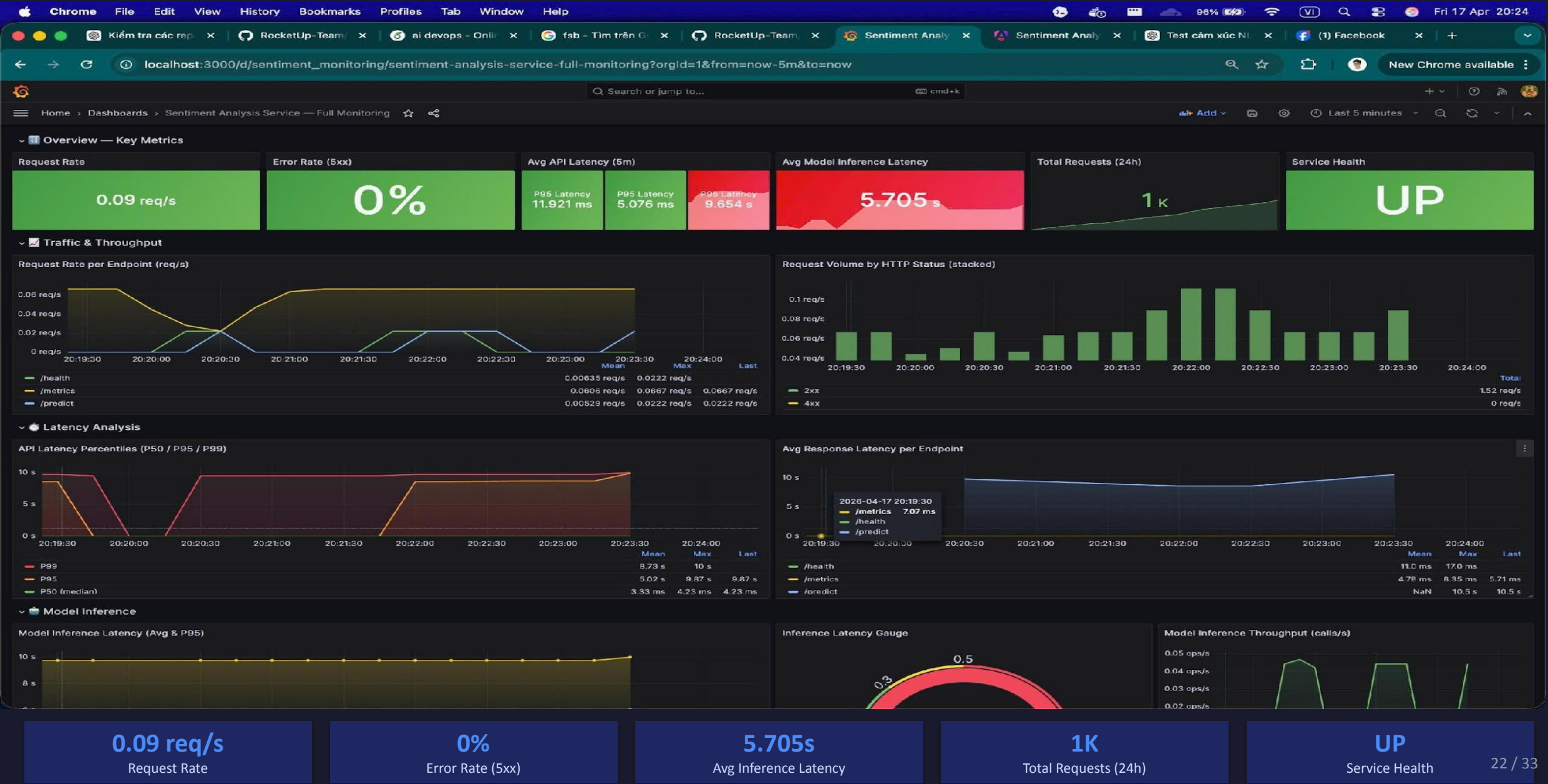
Alerting Rules

HighErrorRate CRITICAL	HTTP 5xx rate > 10% for 1 minute → trigger
HighInferenceLatency WARNING	Avg inference latency > 500ms for 5 minutes → trigger



Grafana — Full Monitoring Dashboard

Grafana — Sentiment Analysis Service Full Monitoring



0.09 req/s
Request Rate

0%
Error Rate (5xx)

5.705s
Avg Inference Latency

1K
Total Requests (24h)

UP
Service Health

06

Responsible AI

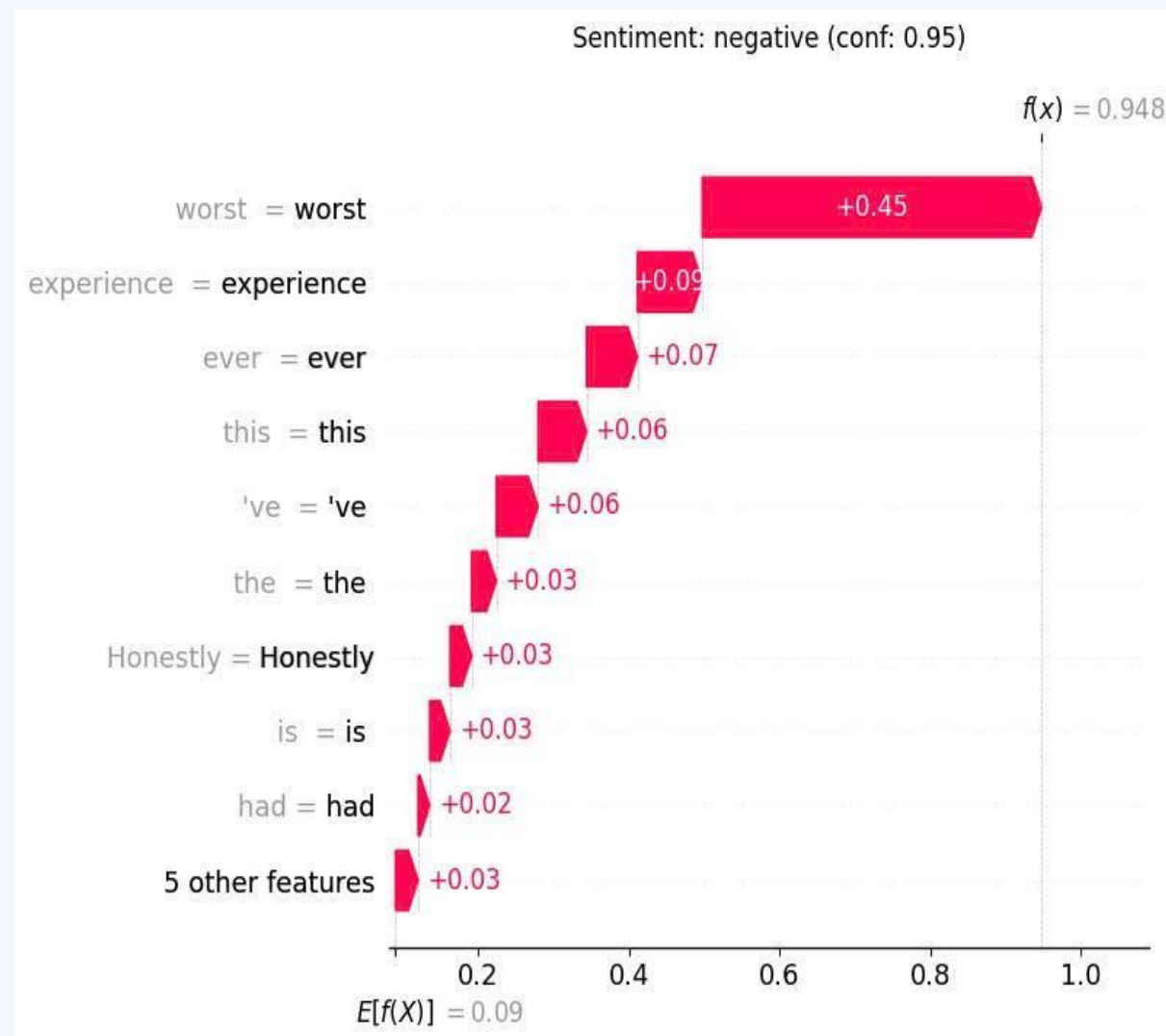
10% — SHAP Explainability, Fairness, Bias, Ethics

SHAP Token-Level Explainability — /explain Endpoint

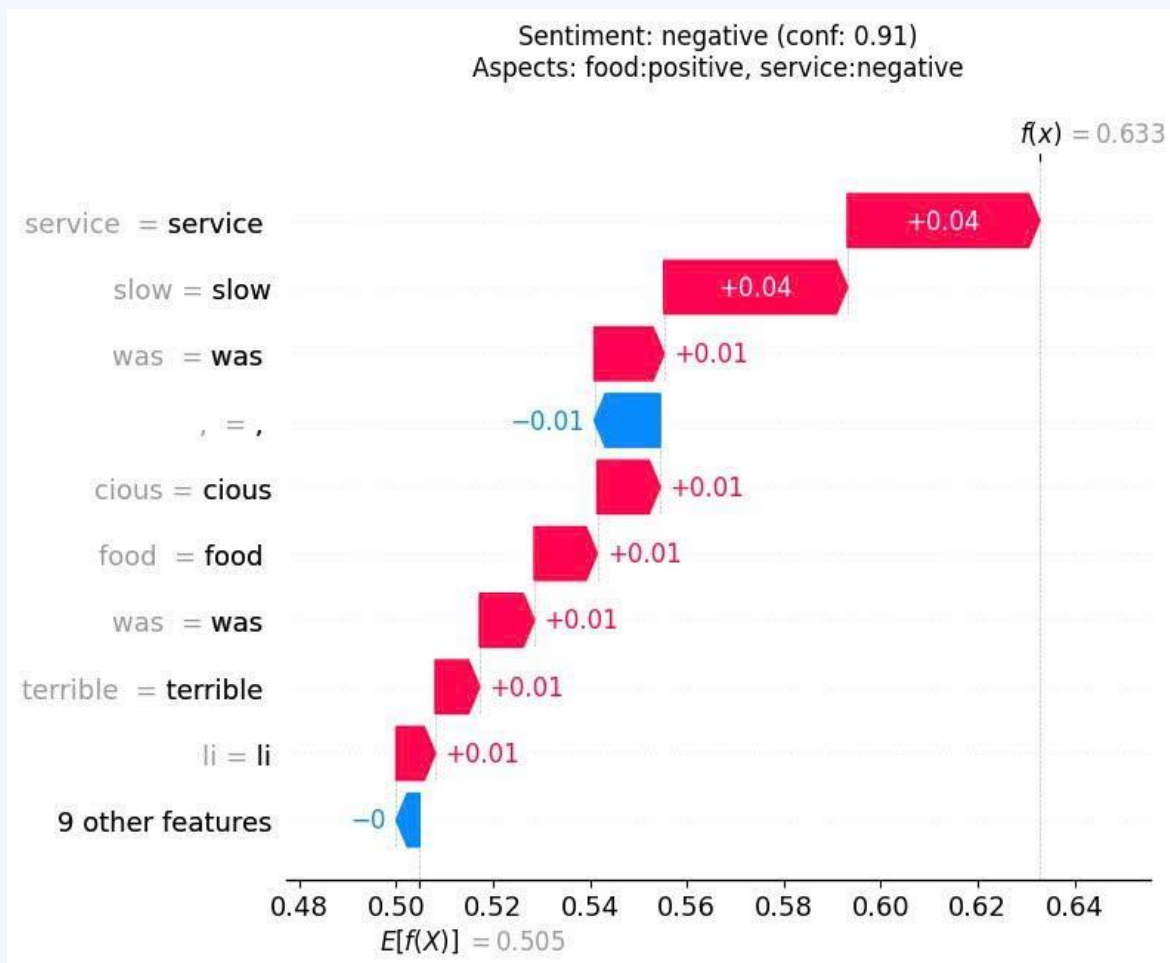
How it works:

- 1 Text is classified → predicted class index determined
- 2 shap.Explainer wraps HuggingFace pipeline
- 3 Shapley values computed by marginalising over token permutations
- 4 Per-token scores extracted for the predicted class
- 5 ExplainResponse enforces $\text{len}(\text{tokens}) == \text{len}(\text{shap_values})$ via Pydantic model_validator

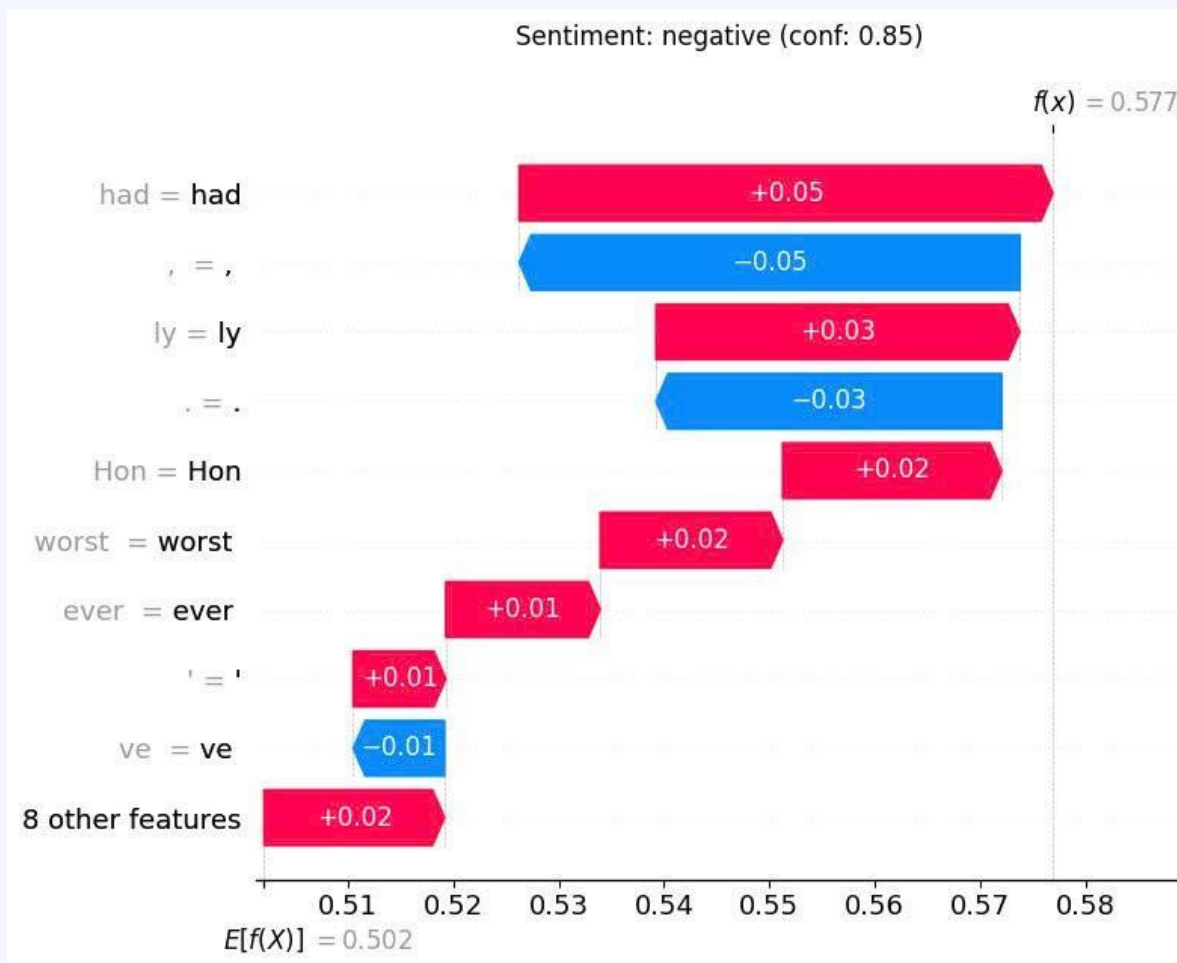
SHAP Waterfall — "Honestly this is the worst experience I've ever had"



SHAP Explanations — Aspect-Aware Attribution



← ABSA: "food:positive, service:negative" (conf 0.91)



← Negative sentiment: "had" strongest feature (conf 0.85) →

Fairness Analysis & Ethical Risk Register

Known Model Biases

Domain Bias

Twitter-trained model — informal social-media language handled better than formal text

Sarcasm Gap

sarcasm_flag field reserved but not populated — known limitation, planned fix

Data Privacy Measures

Stateless service — no user input persisted

No PII in Prometheus logs (method/path/status only)

Model weights inside Docker image — no external registry calls at runtime

Ethical Risk Register

Biased Training Data

Twitter data disadvantages formal text
Mitigation: Future diverse corpus fine-tuning

Acknowledged

Misclassification Harm

Wrong label could influence business decisions
Mitigation: SHAP explanations + human-in-loop

SHAP Done

Sarcasm Detection

Missed sarcasm reverses polarity
Mitigation: Dedicated sarcasm model planned

Planned

Job Displacement

May reduce manual review roles
Mitigation: Positioned as decision-support tool

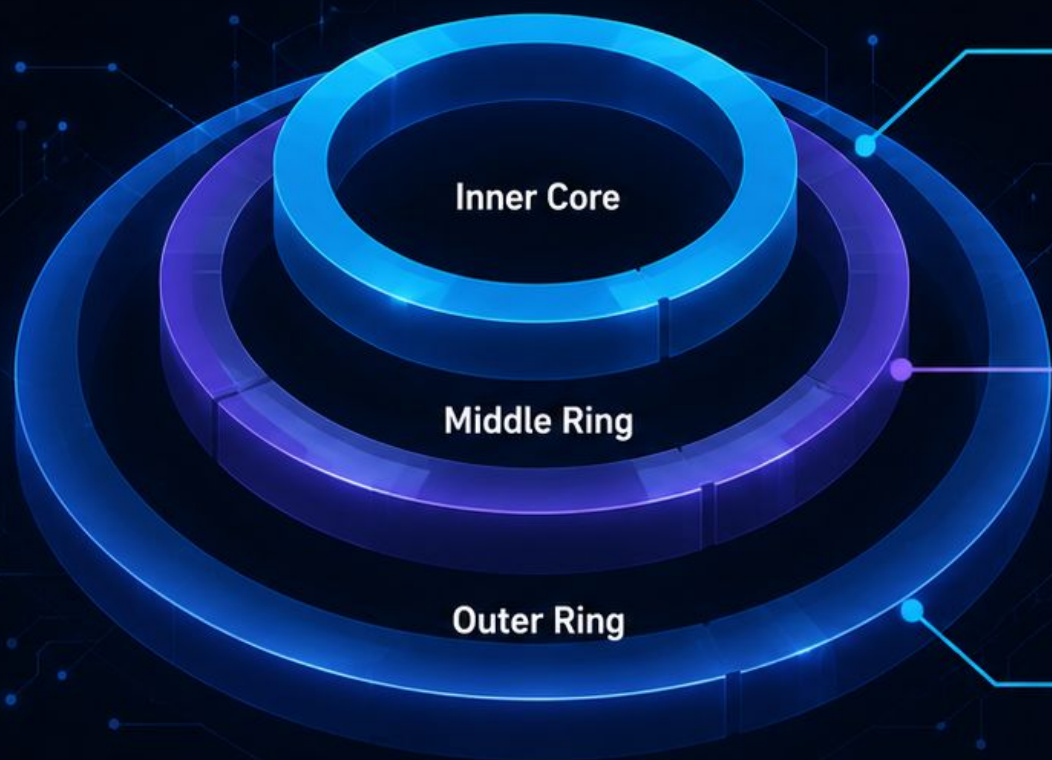
Discussed

07

Testing & Code Quality

15% — Unit, Integration, Data Quality, GitHub Actions

Quality Assurance: 3-Layer System Defense



1

Layer 1: Unit Testing (Data & Model)

Validate data transformation classes (TextCleaner, LengthFilter) using synthetic data. Verify business logic with Pydantic (ensure token counts match SHAP requirements).

2

Layer 2: Integration Testing (API Endpoints)

Use FastAPI TestClient to automatically test all 5 endpoints, ensuring stable data flow from request to response.

3

Layer 3: Data Quality Gates (Validation)

Protect the system from bad data. Check for null-ratio violations ($= 0$) and distribution drift. Alert if critical thresholds are breached.



Reliable

Prevent errors early



Stable

Ensure end-to-end flow



Clean Data

Maintain high data quality



Insightful

Accurate analytics & monitoring

Test Strategy & CI/CD Pipeline

Integration Tests tests/test_api.py

All 5 endpoints tested via FastAPI TestClient
predict, batch_predict, explain, health, metrics

Unit — Data Transforms tests/data/

TextCleaner, LengthFilter, DuplicateRemover, Splitter,
SentimentDeriver — each tested with mock DataFrames

Unit — Model/Contracts tests/model/, tests/contracts/

ModelConfig defaults, schema field constraints,
model_validator logic (len(tokens)==len(shap_values))

GitHub Actions CI/CD Pipeline

Checkout

Fetch source code on ubuntu-latest

Install Deps

pip install -r requirements.txt
(pytest, pytest-cov, httpx)

Test & Coverage

pytest --cov=src tests/
Line coverage measured across src/

Lint (Hard)

flake8 --select=E9,F63,F7,F82
Syntax errors → FAIL build

Lint (Soft)

flake8 --exit-zero --max-complexity=10
Complexity + line-length warnings

08

Results & Live Demo

Performance Metrics · UI Demo · Key Achievements

System Results & Performance Metrics

0%

Error Rate (5xx)

5.7s

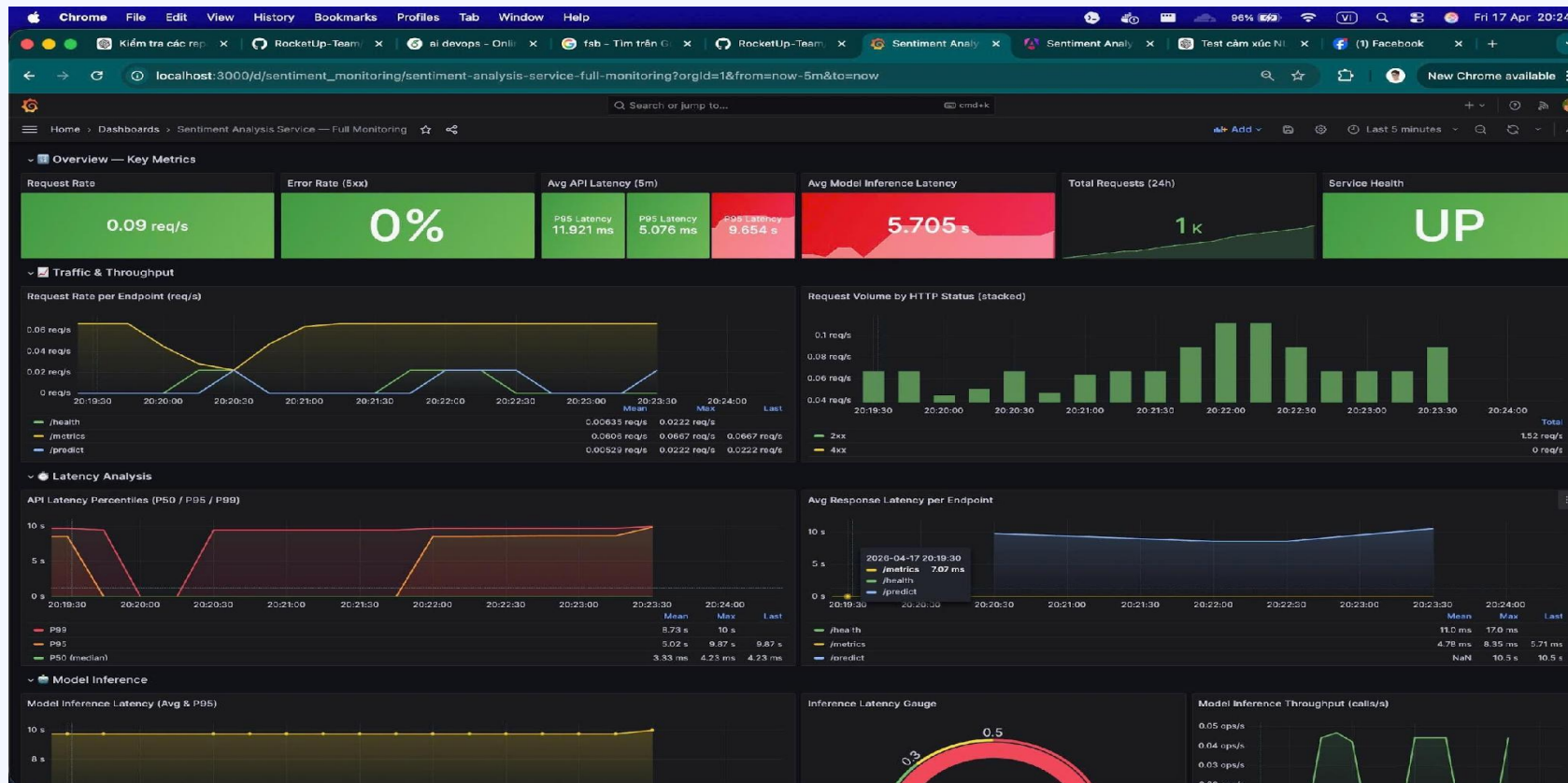
Avg Model Inference

1K+

Total Requests (24h)

UP

Service Health



Key Achievements

- ✓ Full observability stack deployed
- ✓ Zero 5xx errors in testing
- ✓ SHAP explanations working
- ✓ Grafana auto-provisioned
- ✓ DVC pipeline reproducible
- ✓ GitHub Actions CI passing
- ✓ Docker multi-stage image
- ✓ MLflow all runs tracked

Demo Highlights — SentimentPro in Action

SentimentPro

ANALYSIS ENGINE
Active & Ready

SESSION INFO
Total Analysis 20

AI Administrator
SYSTEM ONLINE

Sentiment Analysis
LIVE PREDICTION ACTIVE

NEUTRAL ACCURACY: 88%

Tuyệt vời! Tôi cảm nhận được năng lượng tích cực từ bạn. 🌟
POSITIVE ACCURACY: 100%

Món này thật tuyệt! 🍲
NEGATIVE ACCURACY: 98%

Đêm nay em tuyệt lắm.
POSITIVE ACCURACY: 99%

Thanks for the cold soup and the 1-hour wait, what a wonderful evening!
POSITIVE ACCURACY: 100%

Expensive but worth every penny for the high-quality ingredients they use.
POSITIVE ACCURACY: 99%

Tuyệt vời! Tôi cảm nhận được năng lượng tích cực từ bạn. 🌟
POSITIVE ACCURACY: 99%

Deep analyze sentiment...

Live Test Cases

"This food is very delicious"

POSITIVE

Conf: 100%

"Món này thật tệ"

NEGATIVE

Conf: 98%

"Đêm nay em tuyệt lắm"

POSITIVE

Conf: 99%

"Thanks for the cold soup and
the 1-hour wait..."

POSITIVE

Conf: 100%

"Expensive but worth every penny"

POSITIVE

Conf: 99%

09

Conclusion

Achievements · Lessons Learned · Future Work

Conclusion & Development Roadmap

Short-term (Current)



The system is stable with batch inference, an intuitive UI, and real-time monitoring capabilities.

Mid-term (Operation)



Integrate real user authentication (Auth) and authorization (RBAC).
Deploy a real message queue for batch processing.

Long-term (Expansion)



Continuous Learning.
Detect data drift (Data Drift Detection).
Distribute the system.
Use Kubernetes for auto-scaling.



SentimentPro has successfully transformed the feedback processing pipeline into a smart, transparent, traceable, and measurable data system.



Source code:

GitHub: [RocketUp-Team/Sentiment-Analysis-Service](#)

Conclusion & Future Work

What We Built

✓	Problem Definition	Business-motivated; success metrics at business, model, system levels
✓	System Architecture	Microservices with full observability and experiment tracking
✓	ML (v2)	XLM-RoBERTa + LoRA/PEFT + ONNX FP32/INT8 optimization
✓	Deployment	Multi-stage Docker; REST API with Pydantic; health checks; CORS
✓	Monitoring	3 Prometheus metrics; Grafana auto-provisioned; 2 alert rules
✓	Testing & CI/CD	3 test layers; GitHub Actions lints + tests on every commit
✓	Responsible AI	SHAP via /explain; bias acknowledgment; privacy-preserving; ethics register

Future Work

→ Fine-tune on custom domain data (beyond Twitter corpus)

→ Activate sarcasm_flag with dedicated sarcasm detection model

→ User authentication & rate limiting for production API

→ Kubernetes orchestration for horizontal auto-scaling

→ A/B testing framework for model comparison in production

→ Real-time drift detection with statistical process control

Thank You!

Questions & Discussion

Sentiment Analysis Service · NLP501 Final Project · FPT University / FSB



FastAPI Swagger UI: localhost:8000/docs



Grafana Dashboard: localhost:3000



MLflow Tracking: localhost:5005

Github: <https://github.com/RocketUp-Team/Sentiment-Analysis-Service>